Multi-Agent Embodied Architecture

1. Course Content

Understanding the ROSMASTER-A1 Multi-Agent Embodied Architecture (multi_brains) Framework

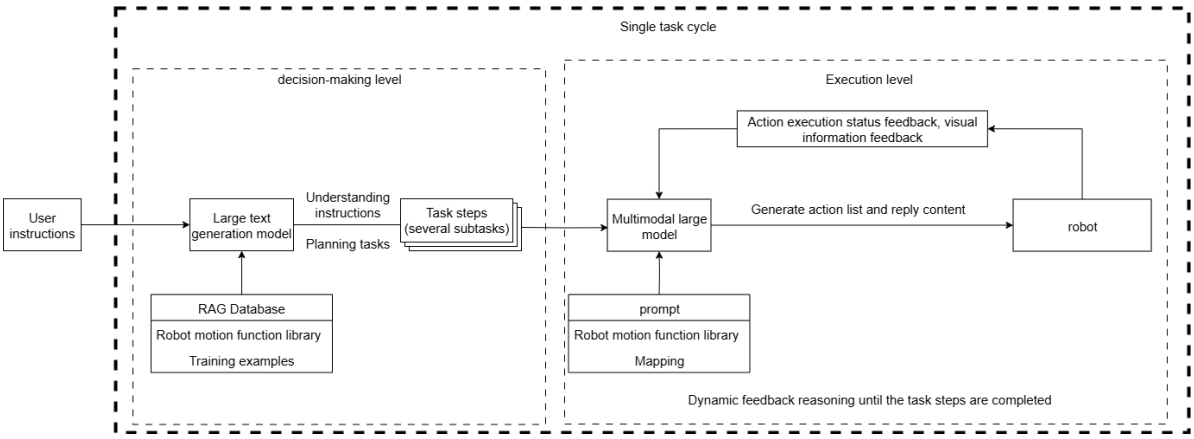
2. Introduction to multi_brains

- multi_brains is an embodied intelligence framework developed by Yabo Intelligence, adapted to general-purpose large AI models. It avoids the problems associated with VLA models and end-to-end models, which require dedicated machines, local computing power, and long data collection and training times. By simply connecting to general-purpose AI models from cloud model providers, edge robots can achieve powerful embodied intelligence effects.
- multi_brains expands training examples through the RAG knowledge base, quickly enabling robots to adapt to different scenario tasks. It can be deployed on most small master control devices, allowing large-scale model inference to be placed in the cloud or on local servers.
- The core idea of multi_brains is to decouple different stages of a task through multiple AI models in a layered manner. This is a comprehensive improvement upon the first-generation self-developed dual-model inference framework. Its main enhancements include:
- A smoother recording experience with inertial tail sound detection and adjustable tail sound detection duration.
- A more flexible decision-making mechanism, adding task routing. The AI agent can autonomously choose between dual-model and single-model inference based on task difficulty, giving the AI greater autonomy and a better user experience.
- An easily understandable visual architecture process. Multi_brains uses Dify to build a visual agent, allowing real-time observation of the AI agent's data flow during debugging, providing users with a more intuitive understanding of the specific process of dual-model inference.
- A fully localized RAG knowledge base, eliminating concerns about data sensitivity.
- Seamless access to most global AI models. Through Dify's model vendor plugins, hundreds of global AI models can be quickly and seamlessly integrated (local models can also be integrated if needed).
- Traceable historical records. By querying Dify's backend access history, users can view the steps and history of all task instruction execution.
- Personalized response voice. Users can customize random response voices. Built-in speech synthesis commands make it simple and easy to use.

- A comprehensive testing toolkit allows for quick verification of core functional modules.
- Simpler onboarding process, streamlining most operations and providing shortcut configurations.

3. Dual-Model Inference Architecture

- The design of **dual-model inference** and **dynamic feedback inference** provides stronger system robustness compared to a single-model architecture.



3.1 Advantages of Dual-Model Inference

3.1.1 Decoupling of Task Decision-Making and Action Transformation

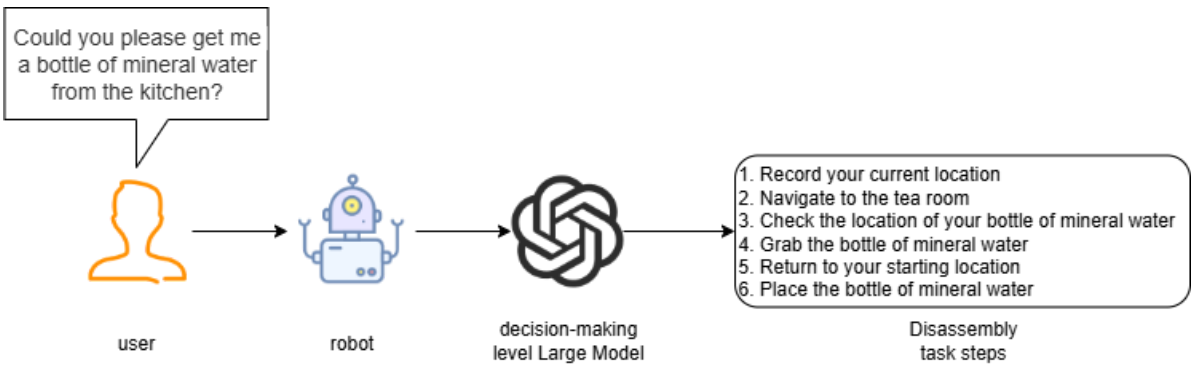
- The decision-making layer AI focuses on thinking, planning, and breaking down task instructions.
- The execution layer AI focuses on chat replies and converting task steps into JSON text.

3.1.2 Improving the Success Rate of Long-Flow Tasks

When using a single model for inference, it is necessary to simultaneously perform "natural language understanding + environmental perception + process decomposition + action function output," which is prone to **modal interference** (language ambiguity or excessively long instructions leading to misunderstandings). The dual-model approach, through phased processing, allows the decision-making layer to focus on task planning, and the execution layer to focus on action execution.

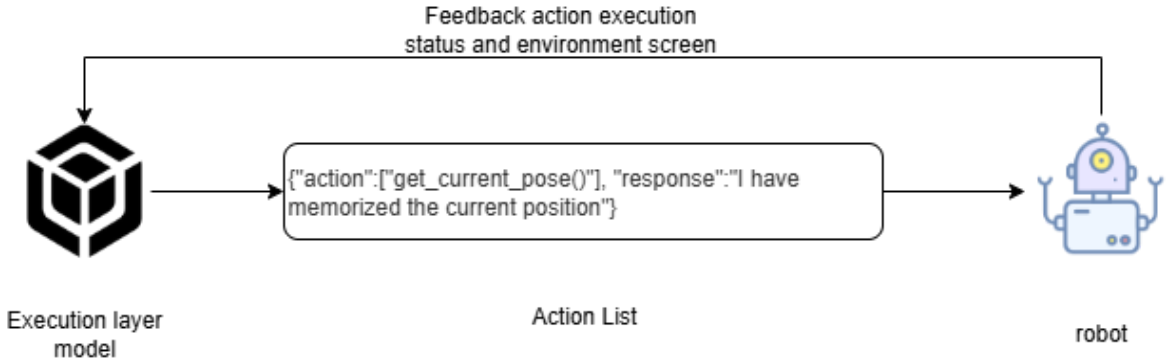
3.2 Decision-Making Layer AI

- The large decision-making layer model is mainly responsible for task planning. It can understand complex human instructions and decompose them into specific task steps. For example, when receiving the instruction "Can you get me a bottle of mineral water from the kitchen?", the large language model will break it down into several tasks, as shown in the diagram below.



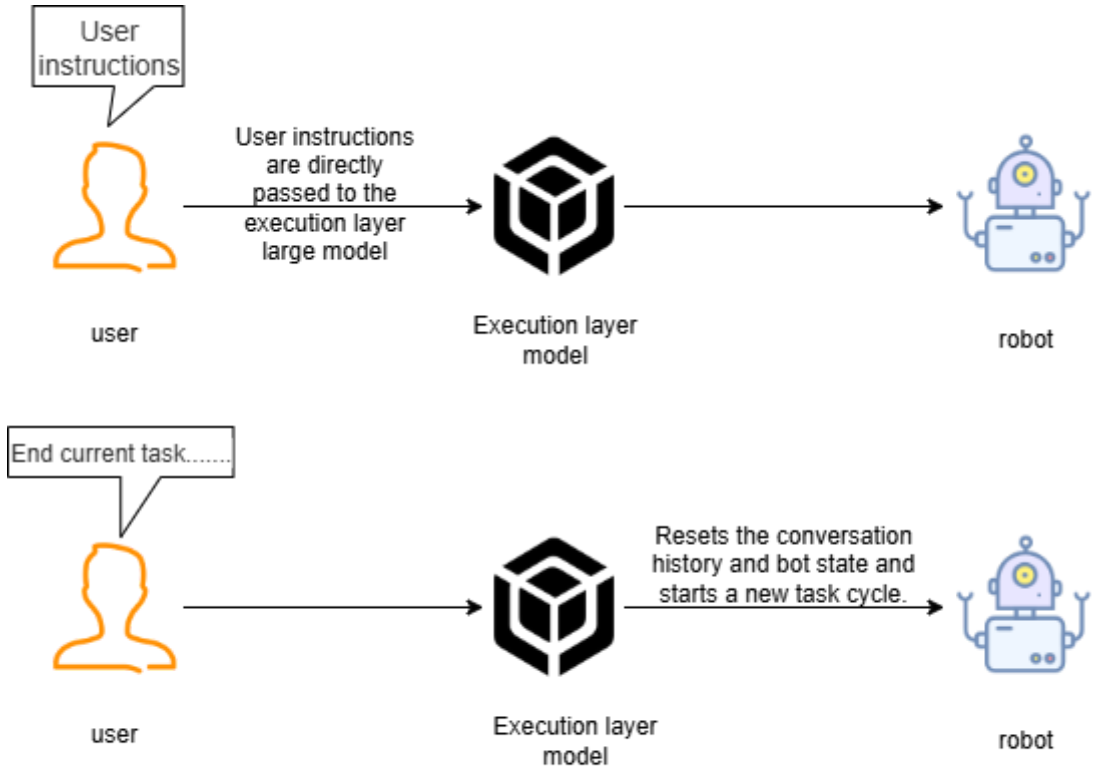
3.3 Execution Layer AI

- The execution layer model is mainly responsible for chat replies, converting task steps into JSON text, continuously receiving feedback (success/failure) and images from the robot's action results, and providing instructions for the next action based on the success or failure of the action execution.
- The execution layer model acts as a supervisor, continuously monitoring the robot's progress in executing task steps, and considering the feedback after the robot's actions and environmental information to determine the next action to be performed, until the task is successfully completed or terminated early due to special circumstances.



4. Task Cycle

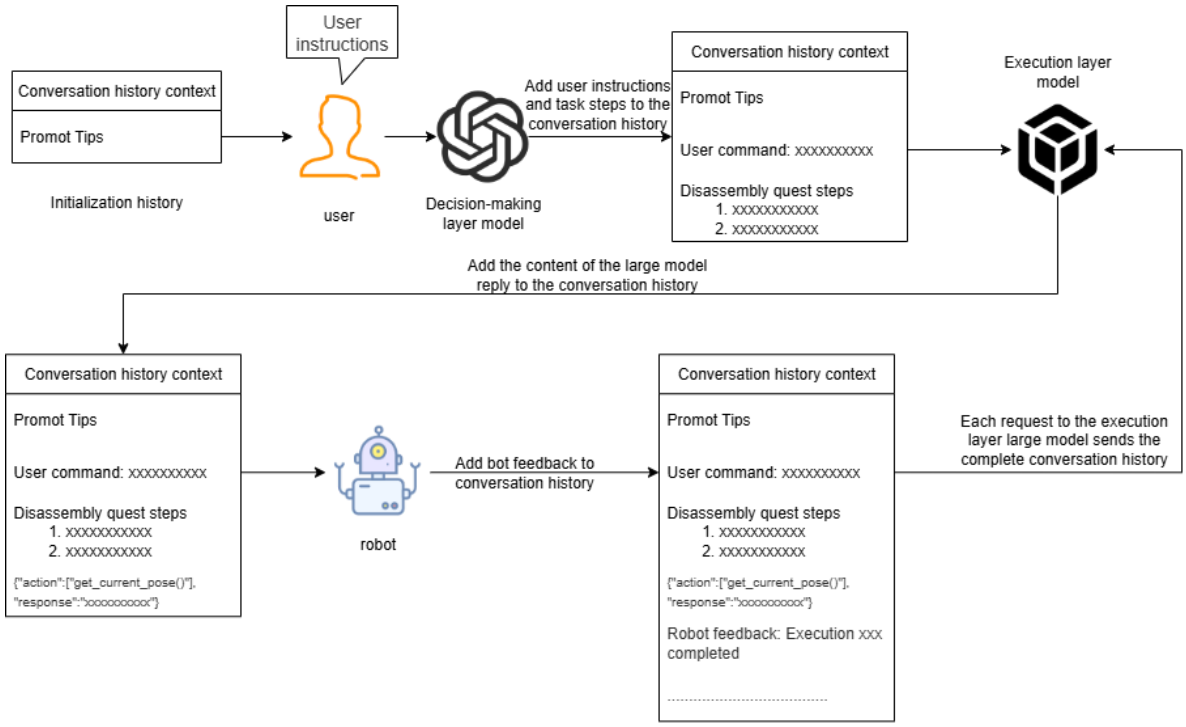
- After waking up the robot, a task cycle begins. All user commands and robot responses are stored in the AI model's historical context, essentially the robot's temporary memory, allowing it to "remember" previous events.
- When the user requests to end the current task and allow the robot to rest, the robot resets the AI model's historical context, essentially causing the robot to "forget" previous events.



The waiting state of the robot after completing the task

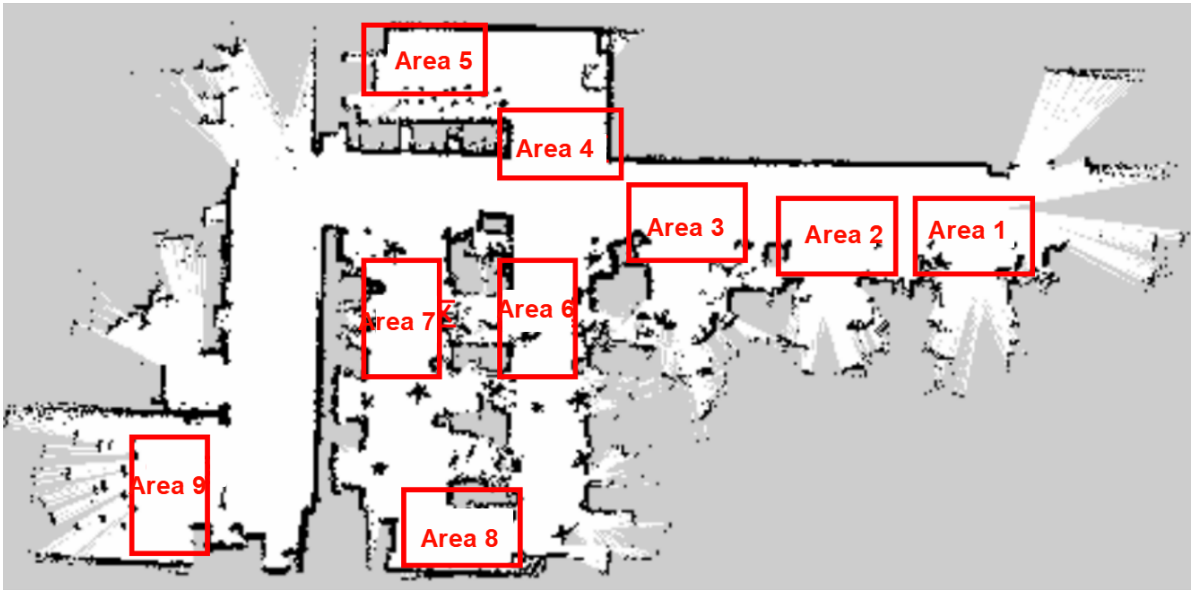
5. Historical Context

- The robot's dialogue history is stored in DIfy's AI application variables, with a default maximum dialogue memory of 50 rounds.



6. Map Mapping

- Robots use grid maps for navigation. If the robot needs to understand its location in the real world, a mapping relationship needs to be established between the grid map and the real-world environment. This relationship is called **map mapping**.
- Suppose we use a robot to generate a **grid map** using SLAM in a factory environment. The real factory has several manually divided areas, as shown in the image below:



We establish a one-to-one correspondence between these real-world areas and letter symbols:

A: "Area 1", B: "Area 2", C: "Area 3", D: "Area 4", E: "Area 5", F: "Area 6", G: "Area 7", H: "Area 8", I: "Area 9"

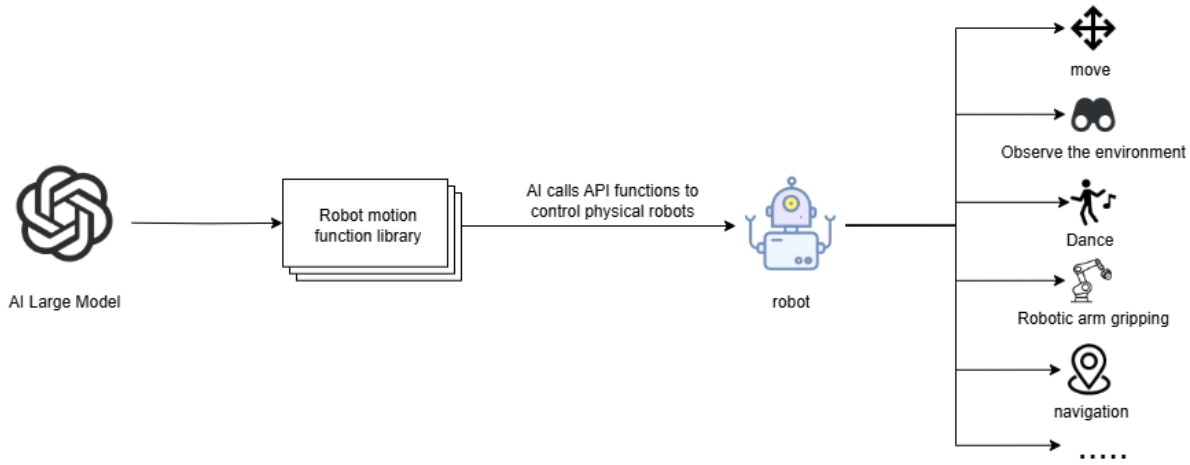
Then we write the map coordinates for the letter symbols in a YAML file, for example:

```
A:
name: 'Area 1'
position:
x: 4.4034953117370605
y: 0.4879316985607147
orientation:
x: 0.0
y: 0.0
z: 0.701498621044694
w: 0.7126708108744126
```

When we ask the robot to go to a certain actual area, we only need to convert the large model into the corresponding letter symbols, so that the robot can understand the location of the area in the real environment.

7. Action Function Library

- The API functions in the robot action function library are the bridge for the large model to control the robot to interact with the real world.
- These API functions define the minimum actions that the physical robot can perform in the physical world.
- The AI large model converts the actions to be performed into JSON text and sends it to the robot, which then parses the JSON text and executes the corresponding actions.



All action functions and their corresponding functionalities are shown in the table below:

Common Functions

Function Name	Parameters	Functionality	Calling Command
move_left(x, angular_speed)	x: rotation angle, angular_speed: angular velocity	Rotate left by x degrees / Rotate counterclockwise one turn	Turn left x degrees
move_right(x, angular_speed)	x: rotation angle, angular_speed: angular velocity	Rotate right by x degrees / Rotate clockwise one turn	Turn right x degrees
set_cmdvel(linear_x, linear_y, angular_z, duration)	linear_x: x-axis linear velocity, linear_y: y-axis linear velocity, angular_z: z-axis angular velocity, duration: topic publishing duration	Control the robot chassis movement by setting linear and angular velocities	Move forward, backward
navigation(x)	x: the symbol corresponding to the target point in the map mapping	Control the robot to navigate to the target point	Navigate to a location
navigation(zero)	zero: the recorded coordinate point	Navigate back to the previously recorded coordinate point	Return to origin, starting position
get_current_pose()	-	Record the current coordinates in the global map to the zero parameter	Remember the current position
seewhat()	-	Take an image of the robot's current view and upload it to the execution layer model	Observe the environment
wait(x)	x: time, unit: seconds	Wait for a period of time without performing any operations	Wait x seconds
finishtask()	-	Automatically called after the robot completes the task, used to stop providing status feedback to the execution layer model	-
finish_dialogue()	-	The robot ends this round of dialogue and starts a new cycle	-

Function Name	Parameters	Function Description	Command Example
face_follow()	-	Face Tracking Function	Start face tracking
qrFollow()	-	QR Code Tracking Function	Start QR code tracking
apriltagFollow()	-	AprilTag Tracking Function	Start AprilTag tracking
color_follow(color)	color values: 'red', 'green', 'blue', 'yellow'	Color Tracking Function	Start yellow tracking
KCF_follow(x1,y1,x2,y2)	Resolution `640×480` pixels, `(x1,y1)` is the top-left corner coordinate of the object's bounding box, `(x2,y2)` is the bottom-right corner coordinate	Object Tracking Function	Start tracking the object in my hand
gestureFollow()	-	Gesture Tracking Function	Start gesture tracking
poseFollow ()	-	Pose Tracking Function	Start pose tracking
stop_follow()	-	Stop all running tracking activities	Stop tracking
follow_line(color)	Automatic line following of the specified color, color values: 'red', 'green', 'blue', 'yellow'	Color Line Following Function	Start following the green line
get_dist(x,y)	Get the distance information of the center coordinates (x,y) of the specified object	Get the distance of the specified object	Please tell me the distance between you and the fan in front of you

Function Name	Parameters	Function Description	Command
servo_nod()	-	Gimbal servo nods	Please nod
servo_shake()	-	Gimbal servo shakes its head	Please shake your head
servo1_move(x)	x represents the angle to rotate	Controls servo 1 to rotate back and forth to the set angle	Servo 1 rotates to x degrees
servo2_move(x)	x represents the angle to rotate	Controls servo 2 to rotate back and forth to the set angle.	Servo 2 rotates to x degrees
servo_init()	-	Initializes gimbal position	Servo initial position, return to initial position, gimbal position reset
apriltagTracker()	-	Gimbal tracking AprilTag function	Start AprilTag tracking, ID code tracking
faceTracker()	-	Gimbal tracking face function	Start face tracking
gestureTracker()	-	Gimbal tracking gesture function	Start gesture tracking

Function Name	Parameters	Function Description	Command
poseTracker()	-	Gimbal tracking pose function	Start human pose tracking
qrTracker()	-	Gimbal tracking QR code function	Start QR code tracking
stop_track()	-	Stop all tracking functions	Stop tracking
colorTrack(color)	Tracks the specified color, color values: 'red', 'green', 'blue', 'yellow'	Color tracking function	Start yellow tracking
monoTracker(x1,y1,x2,y2)	Resolution `640×480` pixels, `(x1,y1)` is the top-left coordinate of the bounding box of the tracked object, `(x2,y2)` is the bottom-right coordinate	Gimbal tracking object	Please track the object in my hand
faceFollow()	-	Face following function	Start face following
qrFollow()	-	QR code following function	Start QR code following
apriltagFollow()	-	AprilTag following function	Start AprilTag following
colorFollow(color)	Follows the specified color, color values: 'red', 'green', 'blue', 'yellow'	Color following function	Start yellow following
poseFollow()	-	Pose following function	Start human pose following
gestureFollow()	-	Gesture following function	Start gesture following

Function Name	Parameters	Function Description	Command
follow_line(color)	Automatically follows the specified color line, color values: 'red', 'green', 'blue', 'yellow'	Color line following function	Start following the green line